

2026-02-24-multi-landing-system

For Claude: *REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.*

Goal: Разделить 4 лендинга (main, banking, realestate, partners) на независимые билды с отдельным деплоем на субдомены.

Architecture: Монорепо с env-переменной `VITE_LANDING`, которая на этапе сборки определяет какой лендинг включить. Каждый билд содержит только свой лендинг + общие компоненты + юридические страницы. Navbar в минимальном режиме для фокусных лендингов. **Tech Stack:** React 19 + Vite 7 + TypeScript, react-router-dom 7, существующий i18n через LanguageContext.

Task 1: Add VITE_LANDING env type

Files:

- Modify: `src/vite-env.d.ts`

Step 1: Update type definition

```
`` typescript
///  
  
interface ImportMetaEnv {  
  readonly VITE_LANG: 'en' | 'ru' | undefined  
  readonly VITE_LANDING: 'main' | 'banking' | 'realestate' | 'partners' | undefined  
}  
  
interface ImportMeta {  
  readonly env: ImportMetaEnv  
}
```

Step 2: Verify no TypeScript errors

Run: `cd "/Users/konstantin/Desktop/Antygravity folder/sync/WTP2" && npx tsc --noEmit`
Expected: No errors (or same errors as before this change).

Task 2: Update vite.config.ts

Files:

- Modify: `vite.config.ts`

```
Step 1: Update config for dynamic outDir and base path `typescript import { defineConfig } from 'vite' import react from '@vitejs/plugin-react'  
const landing = process.env.VITE_LANDING || 'main'  
const lang = process.env.VITE_LANG || 'en'  
  
export default defineConfig({  
  plugins: [react()],  
  base: '/',
```

```

build: {
  outDir: dist/${landing}-${lang} ,
},
define: {
  'import.meta.env.VITE_LANDING': JSON.stringify(landing),
  'import.meta.env.VITE_LANG': JSON.stringify(lang),
},
})
`

```

Важно: base меняется с /WTP2/ на / – каждый субдомен обслуживает сайт от корня. define гарантирует что env-переменные заменяются на литералы при сборке для tree-shaking. **Step 2: Verify dev server still works**

Run: `cd "/Users/konstantin/Desktop/Antygravity folder/sync/WTP2" && npx vite --host --port 5173 &` → open `http://localhost:5173` → should load main landing.

Kill the server after verification.

Task 3: Create landing configuration

Files:

- Create: `src/config/landing.ts`

Step 1: Create the config file `typescript export type LandingId = 'main' | 'banking' | 'realestate' | 'partners'

```

export const LANDING_ID: LandingId =
(import.meta.env.VITE_LANDING as LandingId) || 'main'

export const IS_FOCUSED_LANDING = LANDING_ID !== 'main'

/**
 * Subdomain prefix for the current landing.
 * 'main' → '' (root domain), others → 'banking.' etc.
 */
const SUBDOMAIN_MAP: Record = {
  main: '',
  banking: 'banking.',
  realestate: 'realestate.',
  partners: 'partners.',
}

export const SUBDOMAIN_PREFIX = SUBDOMAIN_MAP[LANDING_ID]

```

```

/**
 * Full domain for the current language version of THIS landing.
 */
export function getDomain(lang: 'en' | 'ru'): string {
  const tld = lang === 'en' ? 'com' : 'ru'
  return `https://${SUBDOMAIN_PREFIX}wtpref.${tld}`
}

```

```

/**
 * Domain for the OTHER language version of THIS landing.
 */
export function getOtherLangDomain(currentLang: 'en' | 'ru'): string {
  const otherLang = currentLang === 'en' ? 'ru' : 'en'
  return getDomain(otherLang)
}

```

```
}
```

```
,
```

Step 2: Verify TypeScript

Run: `cd "/Users/konstantin/Desktop/Antygravity folder/sync/WTP2" && npx tsc --noEmit`

Expected: No new errors.

Task 4: Update LanguageContext for subdomain-aware domains

Files:

- Modify: `src/lib/LanguageContext.tsx`

Step 1: Replace hardcoded domain logic with landing config

Find and replace the domain constants at the top of the file:

```
`typescript // OLD: // const BUILD_LANG: Language = import.meta.env.VITE_LANG === 'ru' ? 'ru' : 'en' // export const
OTHER_DOMAIN = BUILD_LANG === 'en' ? 'https://wtpref.ru' : 'https://wtpref.com' // export const OTHER_LANG_LABEL =
BUILD_LANG === 'en' ? 'Русская версия' : 'English version'
// NEW:
import { getOtherLangDomain } from '../config/landing'

type Language = 'en' | 'ru'

const BUILD_LANG: Language = import.meta.env.VITE_LANG === 'ru' ? 'ru' : 'en'
```

```
export const OTHER_DOMAIN = getOtherLangDomain(BUILD_LANG)
export const OTHER_LANG_LABEL = BUILD_LANG === 'en' ? 'Русская версия' : 'English version'
```

Step 2: Verify TypeScript compiles

Run: `cd "/Users/konstantin/Desktop/Antygravity folder/sync/WTP2" && npx tsc --noEmit`

Expected: No new errors.

Task 5: Refactor App.tsx — conditional routing per landing

Files:

- Modify: `src/App.tsx`

Step 1: Refactor routing to be landing-aware

Replace the entire App.tsx with:

```
`typescript import { lazy, Suspense, type ReactNode } from 'react' import { Routes, Route } from 'react-router-dom'
import Layout from '../components/Layout' import { LANDING_ID } from '../config/landing'
// Shared pages (always included)
const PrivacyPolicyPage = lazy(() => import('../pages/PrivacyPolicyPage'))
const TermsOfServicePage = lazy(() => import('../pages/TermsOfServicePage'))
const SubmitCasePage = lazy(() => import('../pages/SubmitCasePage'))
const ContactPage = lazy(() => import('../pages/ContactPage'))

// Landing-specific imports - only loaded for the matching VITE_LANDING
// Vite replaces LANDING_ID at build time → dead branches are tree-shaken
const MainPages = LANDING_ID === 'main' ? {
```

```

LandingPage: lazy(() => import('./pages/LandingPage')),
ProcessPage: lazy(() => import('./pages/ProcessPage')),
ProductPage: lazy(() => import('./pages/ProductPage')),
PartnersPage: lazy(() => import('./pages/PartnersPage')),
RiskPage: lazy(() => import('./pages/RiskPage')),
EngagementPage: lazy(() => import('./pages/EngagementPage')),
PartnerKitPage: lazy(() => import('./pages/PartnerKitPage')),
UpdatesPage: lazy(() => import('./pages/UpdatesPage')),
TermsPage: lazy(() => import('./pages/TermsPage')),
} : null

```

```

const BankingPage = LANDING_ID === 'banking'
? lazy(() => import('./pages/BankingFirstLandingPage')) : null

```

```

const RealEstatePage = LANDING_ID === 'realestate'
? lazy(() => import('./pages/PostDealLandingPage')) : null

```

```

const PartnersLandingPage = LANDING_ID === 'partners'
? lazy(() => import('./pages/MonetizationLandingPage')) : null

```

```

const PageFallback = (

```

```

Loading...

```

```

)

```

```

function Lazy({ children }: { children: ReactNode }) {
return {children}
}

```

```

function MainRoutes() {
if (!MainPages) return null
const P = MainPages
return (

```

```

<>
} />
} />
} />
} />
} />
} />
} />
} />
} />
} />
} />
} />
} />

```

```

)
}

```

```

function App() {
return (

```

```

}>
{/ Landing-specific index route /}
{LANDING_ID === 'main' && MainRoutes()}
{BankingPage && } />}
{RealEstatePage && } />}
{PartnersLandingPage && } />}

```

```

{/ Shared routes (all landings) /}
} />
} />
} />
} />

```

```

)
}

export default App

```

Логика: LANDING_ID заменяется литералом при сборке. Vite удаляет мертвые ветки и их import() – в финальный бандл попадает только нужный лендинг. **Step 2: Verify TypeScript compiles**
 Run: `cd "/Users/konstantin/Desktop/Antygravity folder/sync/WTP2" && npx tsc --noEmit`
 Expected: No new errors.

Task 6: Create NavbarMinimal for focused landings

Files:

- Create: `src/components/NavbarMinimal.tsx`
- Modify: `src/components/Layout.tsx`

Step 1: Create NavbarMinimal

Минимальный навбар для фокусных лендингов: логотип + СТА-кнопка. Без навигации, без гамбургера.

```

`typescript import React from 'react' import Logo from './Logo' import { useLanguage } from '../lib/LanguageContext'
const NavbarMinimal: React.FC = () => {
const { t } = useLanguage()

return (

```

```

{t\('nav.submitCase'\)}

```

```

)
}

export default NavbarMinimal

```

Step 2: Update Layout.tsx to choose Navbar variant `typescript // Add at the top of Layout.tsx: import { IS_FOCUSED_LANDING } from '../config/landing' import NavbarMinimal from './NavbarMinimal'

```

// In the JSX, replace:
//
// with:
// {IS_FOCUSED_LANDING ? : }

```

Full replacement in Layout.tsx return statement:

```

`typescript return ( <> {t\('a11y.skipToContent'\)} {IS_FOCUSED_LANDING ? : }

```

Step 3: Update hreflang in Layout.tsx

Replace the hreflang useEffect to use landing-aware domains:

```
`typescript import { IS_FOCUSED_LANDING, getDomain } from '../config/landing'
// Replace the existing hreflang useEffect with:
useEffect(() => {
  const otherLang = lang === 'en' ? 'ru' : 'en'
  const selfDomain = getDomain(lang as 'en' | 'ru')
  const otherDomain = getDomain(otherLang)

  const linkSelf = document.createElement('link')
  linkSelf.rel = 'alternate'
  linkSelf.hreflang = lang
  linkSelf.href = `${selfDomain}${pathname}`
  document.head.appendChild(linkSelf)

  const linkOther = document.createElement('link')
  linkOther.rel = 'alternate'
  linkOther.hreflang = otherLang
  linkOther.href = `${otherDomain}${pathname}`
  document.head.appendChild(linkOther)

  return () => {
    document.head.removeChild(linkSelf)
    document.head.removeChild(linkOther)
  }
}, [pathname, lang])
`
```

Step 4: Verify TypeScript compiles

Run: `cd "/Users/konstantin/Desktop/Antygravity folder/sync/WTP2" && npx tsc --noEmit`
Expected: No new errors.

Task 7: Update Footer for focused landings

Files:

- Modify: `src/components/Footer.tsx`

Step 1: Simplify footer for focused landings

Фокусные лендинги не имеют `/process/terms`, поэтому показываем только `privacy` + `ToS` + `cross-lang` link.

```
`typescript // Add import at top: import { IS_FOCUSED_LANDING } from '../config/landing'
// In the legal links section, wrap the delivery terms conditionally:
{!IS_FOCUSED_LANDING && (

  {t('footer.deliveryTerms')}

)}
`
```

Нет других изменений – `OTHER_DOMAIN` и `OTHER_LANG_LABEL` уже обновлены в Task 4 через `LanguageContext`.

Step 2: Verify TypeScript compiles

```
Run: cd "/Users/konstantin/Desktop/Antygravity folder/sync/WTP2" && npx tsc --noEmit
Expected: No new errors.
```

Task 8: Add build scripts to package.json

Files:

- Modify: package.json

```
Step 1: Replace build scripts section `json "scripts": { "dev": "vite", "dev:ru": "VITE_LANG=ru vite", "dev:banking":
"VITE_LANDING=banking vite", "dev:realestate": "VITE_LANDING=realestate vite", "dev:partners": "VITE_LANDING=partners
vite",
"build": "tsc && vite build",

"build:main:en": "VITE_LANDING=main VITE_LANG=en tsc && VITE_LANDING=main VITE_LANG=en vite build",
"build:main:ru": "VITE_LANDING=main VITE_LANG=ru tsc && VITE_LANDING=main VITE_LANG=ru vite build",
"build:banking:en": "VITE_LANDING=banking VITE_LANG=en tsc && VITE_LANDING=banking VITE_LANG=en vite build",
"build:banking:ru": "VITE_LANDING=banking VITE_LANG=ru tsc && VITE_LANDING=banking VITE_LANG=ru vite build",
"build:realestate:en": "VITE_LANDING=realestate VITE_LANG=en tsc && VITE_LANDING=realestate VITE_LANG=en vite
build",
"build:realestate:ru": "VITE_LANDING=realestate VITE_LANG=ru tsc && VITE_LANDING=realestate VITE_LANG=ru vite
build",
"build:partners:en": "VITE_LANDING=partners VITE_LANG=en tsc && VITE_LANDING=partners VITE_LANG=en vite
build",
"build:partners:ru": "VITE_LANDING=partners VITE_LANG=ru tsc && VITE_LANDING=partners VITE_LANG=ru vite
build",

"build:all": "npm run build:main:en && npm run build:main:ru && npm run build:banking:en && npm run
build:banking:ru && npm run build:realestate:en && npm run build:realestate:ru && npm run build:partners:en
&& npm run build:partners:ru",

"preview": "vite preview",
"pdf": "node scripts/generate-pdf.mjs",
"predeploy": "npm run build:main:en",
"deploy": "gh-pages -d dist/main-en"
}
`
```

Важно:

- Каждый build-скрипт устанавливает 05A env: VITE_LANDING + VITE_LANG
- vite.config.ts использует эти env для outDir : dist/main-en , dist/banking-ru и т.д.
- build:all собирает все 8 билдов последовательно
- deploy обновлен: dist-en → dist/main-en
- Старые build:en / build:ru удалены (заменены на build:main:en / build:main:ru)

Step 2: Update .gitignore for new dist structure

Add to .gitignore :

```
dist/
`
```

(Replace old dist-en/ and dist-ru/ entries if present.)

Task 9: Verify all 8 builds

Step 1: Run build:all

Run: `cd "/Users/konstantin/Desktop/Antygravity folder/sync/WTP2" && npm run build:all`

Expected: 8 directories created:

```
`  
dist/main-en/  
dist/main-ru/  
dist/banking-en/  
dist/banking-ru/  
dist/realestate-en/  
dist/realestate-ru/  
dist/partners-en/  
dist/partners-ru/  
`
```

Step 2: Verify each build contains only its landing

For each dist folder, check:

1. `index.html` exists

2. Bundle size is reasonable:

- main-en - largest (has all routes): ~300-400 KB total
- Focused landings - smaller: ~100-200 KB total each

Run:

```
` bash  
cd "/Users/konstantin/Desktop/Antygravity folder/sync/WTP2"  
for dir in dist/*/; do  
echo "=== $dir ==="  
du -sh "$dir"  
ls "$dir"assets/ | wc -l  
done  
`
```

Step 3: Preview each landing

Run (one at a time, verify visually):

```
` bash  
cd "/Users/konstantin/Desktop/Antygravity folder/sync/WTP2"  
npx vite preview --outDir dist/main-en --port 4173  
npx vite preview --outDir dist/banking-en --port 4174  
npx vite preview --outDir dist/realestate-en --port 4175  
npx vite preview --outDir dist/partners-en --port 4176  
`
```

Verify each:

- Landing loads at root /
- /privacy works
- /terms-of-service works
- /submit-case works
- Focused landings have minimal navbar (logo + CTA)
- Main landing has full navbar
- Cross-language link in footer points to correct subdomain

Step 4: Commit

```
`bash git add -A && git commit -m "feat: multi-landing build system (4 landings x 2 languages = 8 builds)"`
```

- Add VITE_LANDING env to select landing at build time
- Conditional routing: each build includes only its landing page
- NavbarMinimal for focused landings (logo + CTA only)
- Subdomain-aware cross-language links and hreflang
- 8 build scripts: build:main:en/ru, build:banking:en/ru, etc.
- base path changed from /WTP2/ to / (for subdomain hosting)

Co-Authored-By: Claude Opus 4.6 " "`

Task 10: Create deploy script for Timeweb (.ru)

Files:

- Create: scripts/deploy-timeweb.sh

Step 1: Create deploy script ``bash #!/bin/bash``

Deploy RU builds to Timeweb via rsync

Usage: `./scripts/deploy-timeweb.sh`

Requires: SSH access to Timeweb configured

```
set -euo pipefail
```

```
TIMWEB_HOST="${TIMWEB_HOST:-}"
```

```
TIMWEB_USER="${TIMWEB_USER:-}"
```

```
if [ -z "$TIMWEB_HOST" ] || [ -z "$TIMWEB_USER" ]; then
```

```
echo "Error: Set TIMWEB_HOST and TIMWEB_USER environment variables"
```

```
echo "Example: TIMWEB_HOST=s123.timeweb.ru TIMWEB_USER=user123 ./scripts/deploy-timeweb.sh"
```

```
exit 1
fi
```

Build all RU versions

```
echo "Building RU versions..." npm run build:main:ru npm run build:banking:ru npm run build:realestate:ru npm run build:partners:ru
```

Deploy each to its subdomain directory

```
echo "Deploying to Timeweb..."
rsync -avz --delete dist/main-ru/ "$TIMEWEB_USER@$TIMEWEB_HOST:~/wtpref.ru/public_html/"
echo " 🟢 wtpref.ru"

rsync -avz --delete dist/banking-ru/ "$TIMEWEB_USER@$TIMEWEB_HOST:~/banking.wtpref.ru/public_html/"
echo " 🟢 banking.wtpref.ru"

rsync -avz --delete dist/realestate-ru/ "$TIMEWEB_USER@$TIMEWEB_HOST:~/realestate.wtpref.ru/public_html/"
echo " 🟢 realestate.wtpref.ru"

rsync -avz --delete dist/partners-ru/ "$TIMEWEB_USER@$TIMEWEB_HOST:~/partners.wtpref.ru/public_html/"
echo " 🟢 partners.wtpref.ru"

echo ""
echo "🟢 All RU sites deployed to Timeweb!"
```

Step 2: Make executable `bash chmod +x scripts/deploy-timeweb.sh` **Note:** Пути на Timeweb (~/wtpref.ru/public_html/) могут отличаться — уточнить при настройке хостинга.

Что НЕ входит в этот план (делается через веб-интерфейсы)

1. **Cloudflare Pages:** Создание 4 проектов, привязка доменов, настройка build commands — делается через dashboard.cloudflare.com
2. **Timeweb:** Создание 4 субдоменов, настройка DNS — делается через панель Timeweb
3. **DNS .com:** Перенос NS на Cloudflare
4. **GA4:** Создание Data Stream per subdomain
5. **SSL:** Автоматически через Cloudflare / Let's Encrypt

Порядок после реализации кода

1. Код готов (Tasks 1-9) → push в GitHub
2. Настроить Cloudflare Pages (4 проекта для .com)
3. Настроить субдомены в Timeweb (4 для .ru)
4. Запустить deploy-timeweb.sh
5. Проверить все 8 URL
6. Настроить GA4 Data Streams